ORBIT · PYTHON · PAPER 04

第 4 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

关于Python的多态性，以下说法正确的是：

A. 多态只能通过继承实现
B. Python通过鸭子类型实现多态
C. 多态需要显式声明
D. Python不支持多态

答案 B. Python通过鸭子类型实现多态

解析 Python通过鸭子类型（duck typing）实现多态，不需要继承或显式声明。

示例：

```
# 鸭子类型：如果它走起来像鸭子，叫起来像鸭子，那它就是鸭子
class Duck:
    def quack(self):
        return "Quack!"
    def fly(self):
        return "Duck flying"
class Person:
    def quack(self):
        return "I'm imitating a duck!"
    def fly(self):
        return "Person can't fly"
def make_it_quack(duck_like):
    # 不检查类型，只要有quack方法就行
    print(duck_like.quack())
duck = Duck()
person = Person()
make_it_quack(duck)
# Quack!
make_it_quack(person)
# I'm imitating a duck!
```

知识拓展：鸭子类型vs传统多态：

```
# 传统面向对象（Java风格）
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "Woof"
class Cat(Animal):
    def make_sound(self):
        return "Meow"
def animal_sound(animal: Animal):
    print(animal.make_sound())
# Python鸭子类型（更灵活）
class Robot:
    def make_sound(self):
```

```
        return "Beep"
# Robot不继承Animal，但也能使用
animal_sound(Dog())
# Woof
animal_sound(Robot())
# Beep - 鸭子类型允许
```

02 • 单项选择

以下代码的输出是什么？

```
class A:
    def method(self):
        return "A"
class B(A):
    def method(self):
        return "B"
obj = B()
print(obj.method())
```

A. A
B. B
C. AB
D. 报错

答案 B. B

解析 子类重写（override）了父类方法，调用时执行子类的实现。

示例：

```
class Animal:
    def speak(self):
        return "Some sound"
class Dog(Animal):
    def speak(self):
        # 重写父类方法
        return "Woof"
class Cat(Animal):
    def speak(self):
        # 重写父类方法
        return "Meow"
dog = Dog()
cat = Cat()
print(dog.speak())
# Woof
print(cat.speak())
# Meow
# 多态性：同一方法，不同行为
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.speak())
# 输出：
# Woof
# Meow
# Some sound
```

知识拓展：调用父类方法：

```
class Parent:
    def method(self):
        return "Parent"
class Child(Parent):
    def method(self):
        parent_result = super().method()
        # 调用父类方法
        return f"{parent_result} + Child"
obj = Child()
print(obj.method())
# Parent + Child
```

03 • 单项选择

关于 @classmethod 装饰器，以下说法正确的是：

A. 第一个参数是 self

B. 第一个参数是 cls

C. 不能访问类变量

D. 只能通过实例调用

答案 B. 第一个参数是 cls

解析 @classmethod 装饰器的第一个参数是 cls ，代表类本身。

示例：

```
class MyClass:
    count = 0
    def __init__(self, name):
        self.name = name
        MyClass.count += 1
    @classmethod
    def get_count(cls):
        # cls代表类本身
        return cls.count
    @classmethod
    def create_default(cls):
        # 可以用cls创建实例
        return cls("default")
obj1 = MyClass("A")
obj2 = MyClass("B")
print(MyClass.get_count())
# 2 - 类方法可以通过类调用
print(obj1.get_count())
# 2 - 也可以通过实例调用
obj3 = MyClass.create_default()
print(obj3.name)
# default
```

三种方法类型对比：

```
class Example:
    class_var = "class variable"
    def instance_method(self):
        # 实例方法：第一个参数是self
        return f"Instance: {self}"
```

```
    @classmethod
    def class_method(cls):
        # 类方法：第一个参数是cls
        return f"Class: {cls.__name__}, {cls.class_var}"
    @staticmethod
    def static_method(x, y):
        # 静态方法：没有特殊的第一个参数
        return x + y
obj = Example()
print(obj.instance_method())
# Instance: <__main__.Example object>
print(Example.class_method())
# Class: Example, class variable
print(Example.static_method(1, 2))
# 3
```

知识拓展：类方法的应用场景：

- 工厂方法（alternative constructors）
- 访问/修改类变量
- 实现单例模式

04 • 单项选择

以下哪个魔法方法用于实现对象的比较运算？

A. `__cmp__()`

B. `__compare__()`

C. `__eq__()`

D. `__is__()`

答案 C. `__eq__()`

解析 `__eq__()` 魔法方法用于实现相等比较（ == 运算符）。

示例：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        # 实现 == 运算符
        if not isinstance(other, Point):
            return False
        return self.x == other.x and self.y == other.y
    def __ne__(self, other):
        # 实现 != 运算符
        return not self.__eq__(other)
p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)
print(p1 == p2)
# True
print(p1 == p3)
# False
print(p1 != p3)
# True
```

所有比较魔法方法：

```
class Number:
    def __init__(self, value):
        self.value = value
    def __eq__(self, other):
        # ==
        return self.value == other.value
    def __ne__(self, other):
        # !=
        return self.value != other.value
    def __lt__(self, other):
        # <
        return self.value < other.value
    def __le__(self, other):
        # <=
        return self.value <= other.value
    def __gt__(self, other):
        # >
        return self.value > other.value
    def __ge__(self, other):
        # >=
        return self.value >= other.value
n1 = Number(5)
n2 = Number(10)
print(n1 < n2)
# True
print(n1 <= n2)
# True
print(n1 == n2)
# False
print(n1 > n2)
# False
```

知识拓展：使用 functools.total_ordering 简化：

```
from functools import total_ordering
@total_ordering
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade
    def __eq__(self, other):
        return self.grade == other.grade
    def __lt__(self, other):
        return self.grade < other.grade
# 只需定义__eq__和__lt__，其他比较方法自动生成
s1 = Student("Alice", 85)
s2 = Student("Bob", 90)
print(s1 < s2)
# True
print(s1 <= s2)
# True - 自动生成
print(s1 > s2)
```

```
# False - 自动生成
```

05 • 单项选择

关于Python的抽象基类，以下说法正确的是：

A. 使用 abc 模块
B. 使用 @abstractmethod 装饰器
C. 抽象类不能被实例化
D. 以上都正确

答案 D. 以上都正确

解析 所有选项都正确，抽象基类使用 abc 模块、 @abstractmethod 装饰器，且不能被实例化。

示例：

```
from abc import ABC, abstractmethod
# A. 使用abc模块
class Shape(ABC):
    # 继承ABC
    # B. 使用@abstractmethod装饰器
    @abstractmethod
    def area(self):
        pass
    @abstractmethod
    def perimeter(self):
        pass
    def describe(self):
        # 可以有具体方法
        return f"Area: {self.area()}, Perimeter: {self.perimeter()}"
# C. 抽象类不能被实例化
try:
    shape = Shape()
except TypeError as e:
    print(f"错误: {e}")
# 错误: Can't instantiate abstract class Shape with abstract methods area, perimeter
# 子类必须实现所有抽象方法
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)
rect = Rectangle(5, 3)
print(rect.describe())
                # Area: 15, Perimeter: 16
```

知识拓展：抽象属性和类方法：

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @property
    @abstractmethod
    def name(self):
```

```
            # 抽象属性
            pass
        @classmethod
        @abstractmethod
        def category(cls):
            # 抽象类方法
            pass
        @staticmethod
        @abstractmethod
        def description():
            # 抽象静态方法
            pass
    class Dog(Animal):
        @property
        def name(self):
            return "Dog"
        @classmethod
        def category(cls):
            return "Mammal"
        @staticmethod
        def description():
            return "A domestic animal"
    dog = Dog()
    print(dog.name)
    # Dog
    print(Dog.category())
    # Mammal
    print(Dog.description())
                    # A domestic animal
```

06 • 单项选择

以下代码的输出是什么？

```
class MyClass:
    count = 0
    def __init__(self):
        MyClass.count += 1
    @classmethod
    def get_count(cls):
        return cls.count
a = MyClass()
b = MyClass()
print(MyClass.get_count())
```

A. 0
B. 1
C. 2
D. 报错

答案 C. 2

解析 每次创建实例时， `__init__` 执行， `count` 递增，创建两个实例后count为2。

示例：

```
class Counter:
    count = 0
    # 类变量
    def __init__(self):
        Counter.count += 1
        # 每次实例化时递增
        self.instance_id = Counter.count
c1 = Counter()
print(f"c1 ID: {c1.instance_id}, Total: {Counter.count}")
# c1 ID: 1, Total: 1
c2 = Counter()
print(f"c2 ID: {c2.instance_id}, Total: {Counter.count}")
# c2 ID: 2, Total: 2
c3 = Counter()
print(f"c3 ID: {c3.instance_id}, Total: {Counter.count}")
# c3 ID: 3, Total: 3
类变量vs实例变量：
class Example:
    class_var = 0
    # 类变量，所有实例共享
    def __init__(self, value):
        self.instance_var = value
        # 实例变量，每个实例独立
        Example.class_var += 1
e1 = Example(10)
e2 = Example(20)
print(f"e1.instance_var: {e1.instance_var}")
# 10
print(f"e2.instance_var: {e2.instance_var}")
# 20
print(f"class_var: {Example.class_var}")
# 2
# 修改类变量影响所有实例
Example.class_var = 100
print(f"e1通过类访问: {Example.class_var}")
# 100
print(f"e2通过类访问: {Example.class_var}")
# 100
知识拓展：类变量的陷阱：
class Trap:
    data = []
    # 类变量，共享！
    def add(self, item):
        self.data.append(item)
t1 = Trap()
t2 = Trap()
t1.add(1)
t2.add(2)
print(t1.data)
# [1, 2] - 意外！
print(t2.data)
```

```
# [1, 2] - 共享同一个列表
# 正确做法：使用实例变量
class Correct:
    def __init__(self):
        self.data = []
    # 每个实例独立的列表
    def add(self, item):
        self.data.append(item)
c1 = Correct()
c2 = Correct()
c1.add(1)
c2.add(2)
print(c1.data)
# [1]
print(c2.data)
# [2] - 正确
```

07 • 单项选择

关于生成器的 close() 方法，以下说法正确的是：

A. 用于关闭生成器

B. 会触发 GeneratorExit 异常

C. 之后不能再使用该生成器

D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是生成器的正确特性。

示例：

```
# A. 使用yield关键字
def simple_generator():
    yield 1
    yield 2
    yield 3
gen = simple_generator()
print(next(gen))
# 1
print(next(gen))
# 2
# B. 惰性求值
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1
gen = infinite_sequence()
print(next(gen))
# 0
print(next(gen))
# 1
# 不会无限循环，因为是惰性求值
# C. 节省内存
import sys
```

```
# 列表推导式：立即创建所有元素
list_comp = [x**2 for x in range(1000000)]
print(f"列表大小: {sys.getsizeof(list_comp)} bytes")
# 生成器表达式：按需生成
gen_exp = (x**2 for x in range(1000000))
print(f"生成器大小: {sys.getsizeof(gen_exp)} bytes")
# 生成器远小于列表
# D. 只能遍历一次
gen = (x for x in range(3))
print(list(gen))
# [0, 1, 2]
print(list(gen))
                # [] - 已耗尽
知识拓展：生成器的高级用法：
# 生成器表达式
gen = (x**2 for x in range(5))
# 使用yield from委托
def delegate_generator():
    yield from range(3)
    yield from ['a', 'b', 'c']
for item in delegate_generator():
    print(item)
# 0, 1, 2, a, b, c
# 生成器的send()方法
def echo_generator():
    while True:
        received = yield
        print(f"Received: {received}")
gen = echo_generator()
next(gen)
# 启动生成器
gen.send("Hello")
# Received: Hello
gen.send("World")
# Received: World
```

08 • 单项选择

以下代码的输出是什么？

```
def gen():
    try:
        yield 1
        yield 2
    finally:
        print("清理")
g = gen()
next(g)
g.close()
```

A. 只输出1

B. 输出1和"清理"

C. 输出1、2和"清理"

D. 报错

答案 B. 输出1和"清理"

解析 try块执行并返回1，finally块总是执行并打印"清理"。

示例：

```
def test_finally():
    try:
        print("try块")
        return 1
    finally:
        print("清理")
# 即使有return，finally也会执行
result = test_finally()
print(f"返回值: {result}")
# 输出：
# try块
# 清理
# 返回值: 1
finally的各种情况：
# 1. 正常执行
def case1():
    try:
        return "normal"
    finally:
        print("cleanup")
print(case1())
# cleanup
# normal
# 2. 异常发生
def case2():
    try:
        raise ValueError("error")
    except ValueError:
        return "caught"
    finally:
        print("cleanup")
print(case2())
# cleanup
# caught
# 3. finally中的return会覆盖try中的return
def case3():
    try:
        return "from try"
    finally:
        return "from finally"
# 不推荐！
print(case3())
# from finally
# 4. finally总是执行，即使break/continue
for i in range(3):
```

```
        try:
            if i == 1:
                continue
            print(f"i = {i}")
        finally:
            print(f"cleanup for {i}")
# 输出：
# i = 0
# cleanup for 0
# cleanup for 1
# i = 2
# cleanup for 2
知识拓展：try-except-else-finally的完整用法：
def complete_example(x):
    try:
        result = 10 / x
    except ZeroDivisionError:
        print("除零错误")
        return None
    else:
        # 只在没有异常时执行
        print("计算成功")
        return result
    finally:
        # 总是执行
        print("清理资源")
print(complete_example(2))
# 计算成功
# 清理资源
# 5.0
print(complete_example(0))
# 除零错误
# 清理资源
# None
```

09 • 单项选择

关于迭代器和生成器的关系，以下说法正确的是：

A. 生成器是特殊的迭代器

B. 迭代器是特殊的生成器

C. 两者没有关系

D. 完全相同

答案 A. 生成器是特殊的迭代器

解析 生成器是实现了迭代器协议（ `__iter__` 和 `__next__` ）的特殊迭代器。

示例：

```
# 生成器自动实现迭代器协议
def my_generator():
    yield 1
    yield 2
    yield 3
gen = my_generator()
```

```
# 生成器有__iter__和__next__方法
print(hasattr(gen, '__iter__'))
# True
print(hasattr(gen, '__next__'))
# True
# 可以手动调用__next__
print(gen.__next__())
# 1
print(gen.__next__())
# 2
# 也可以用next()函数
print(next(gen))
# 3
# 耗尽后抛出StopIteration
try:
    next(gen)
except StopIteration:
    print("生成器耗尽")
```

生成器vs手动实现的迭代器：

```
# 手动实现迭代器
class ManualIterator:
    def __init__(self, max_num):
        self.max_num = max_num
        self.current = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.current < self.max_num:
            self.current += 1
            return self.current
        raise StopIteration
# 使用生成器（更简洁）
def generator_version(max_num):
    current = 0
    while current < max_num:
        current += 1
        yield current
# 两者功能相同
for i in ManualIterator(3):
    print(i)
# 1, 2, 3
for i in generator_version(3):
    print(i)
# 1, 2, 3
```

知识拓展：生成器的内部状态：

```
import inspect
def stateful_generator():
    print("开始")
    yield 1
    print("中间")
    yield 2
```

```
    print("结束")
    yield 3
gen = stateful_generator()
# 查看生成器状态
print(inspect.getgeneratorstate(gen))
# GEN_CREATED
next(gen)
# 开始
print(inspect.getgeneratorstate(gen))
# GEN_SUSPENDED
next(gen)
# 中间
next(gen)
# 结束
try:
    next(gen)
except StopIteration:
    pass
print(inspect.getgeneratorstate(gen))
# GEN_CLOSED
```

10 • 单项选择

以下代码的输出是什么？

```
it = iter([1, 2, 3])
print(next(it))
```

A. 1 2 3 None

B. 1 2 3 StopIteration

C. 1 2 3 然后抛出StopIteration异常

D. 1 2 3 0

答案 C. 1 2 3 然后抛出StopIteration异常

解析 生成器前三次调用正常输出0、1、2，第四次调用抛出 StopIteration 异常。

示例：

```
def count_to_three():
    for i in range(3):
        yield i
gen = count_to_three()
print(next(gen))
# 0
print(next(gen))
# 1
print(next(gen))
# 2
try:
    print(next(gen))
# 第4次调用
except StopIteration:
    print("生成器已耗尽")
# 输出：生成器已耗尽
for循环自动处理
```

循环自动处理StopIteration：

```
def my_range(n):
    i = 0
    while i < n:
        yield i
        i += 1
# for循环自动捕获StopIteration
for num in my_range(3):
    print(num)
# 0, 1, 2 - 自动停止，不会抛出异常
# 手动迭代需要处理异常
gen = my_range(3)
while True:
    try:
        print(next(gen))
    except StopIteration:
        print("结束")
        break
```

知识拓展：StopIteration的值：

```
def generator_with_return():
    yield 1
    yield 2
    return "Done"
# return的值会传递给StopIteration
gen = generator_with_return()
print(next(gen))
# 1
print(next(gen))
# 2
try:
    next(gen)
except StopIteration as e:
    print(f"StopIteration值: {e.value}")
# Done
# 在协程中使用
def delegating_generator():
    result = yield from generator_with_return()
    print(f"收到返回值: {result}")
    return result
gen = delegating_generator()
print(next(gen))
# 1
print(next(gen))
# 2
try:
    next(gen)
except StopIteration as e:
    print(f"最终返回: {e.value}")
# 收到返回值: Done
# 最终返回: Done
```

11 • 单项选择

关于装饰器的参数，以下说法正确的是：

A. 装饰器不能有参数
B. 带参数的装饰器需要多层嵌套
C. 装饰器参数只能是字符串
D. 装饰器参数必须有默认值

答案 B. 带参数的装饰器需要多层嵌套

解析 带参数的装饰器需要三层函数：最外层接收参数，中间层是真正的装饰器，最内层是包装函数。

示例：

```
# 带参数的装饰器：三层结构
def repeat(times):
    # 第1层：接收装饰器参数
    def decorator(func):
        # 第2层：真正的装饰器
        def wrapper(*args, **kwargs):
            # 第3层：包装函数
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
@repeat(3)
# 传入参数
def say_hello():
    print("Hello!")
say_hello()
# 打印3次Hello!
# Hello!
```

无参数vs有参数装饰器对比：

```
# 无参数装饰器：两层结构
def simple_decorator(func):
    def wrapper(*args, **kwargs):
        print("函数执行前")
        result = func(*args, **kwargs)
        print("函数执行后")
        return result
    return wrapper
@simple_decorator
def func1():
    print("函数1")
# 有参数装饰器：三层结构
def parameterized_decorator(prefix):
    def decorator(func):
        def wrapper(*args, **kwargs):
            print(f"{prefix}: 函数执行前")
            result = func(*args, **kwargs)
            print(f"{prefix}: 函数执行后")
            return result
        return wrapper
```

```
        return decorator
    @parameterized_decorator("LOG")
    def func2():
        print("函数2")
    func2()
    # LOG: 函数执行前
    # 函数2
    # LOG: 函数执行后
    知识拓展：可选参数的装饰器：
    import functools
    def smart_decorator(arg=None):
        def decorator(func):
            @functools.wraps(func)
            def wrapper(*args, **kwargs):
                if arg:
                    print(f"Arg: {arg}")
                return func(*args, **kwargs)
            return wrapper
        # 判断是否直接作为装饰器使用
        if callable(arg):
            return decorator(arg)
    # 无参数使用
    @smart_decorator
    def func1():
        print("func1")
    # 有参数使用
    @smart_decorator(arg="custom")
    def func2():
        print("func2")
    func1()
    # func1
    func2()
    # Arg: custom\nfunc2
```

12 • 单项选择

以下代码的输出是什么？

```
def repeat(n):
    def decorator(func):
        def wrapper(*args):
            for _ in range(n):
                func(*args)
        return wrapper
    return decorator
@repeat(3)
def say(msg):
    print(msg)
say("Hello")
```

A. Hello

B. Hello Hello Hello

C. 3

D. 报错

答案 B. Hello Hello Hello

解析 带参数的装饰器 @repeat(3) 使函数执行3次，每次调用都打印"Hello"。

示例：

```
def repeat(n):
    def decorator(func):
        def wrapper(*args):
            for _ in range(n):
                func(*args)
        return wrapper
    return decorator
@repeat(3)
def say(msg):
    print(msg)
say("Hello")
# 输出：
# Hello
```

知识拓展：带返回值的重复装饰器

```
def repeat_with_return(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            result = None
            for i in range(n):
                result = func(*args, **kwargs)
                if i < n - 1:
                print(f"第{i+1}次执行")
            return result
        # 返回最后一次的结果
        return wrapper
    return decorator
@repeat_with_return(3)
def calculate(x, y):
    result = x + y
    print(f"计算结果: {result}")
    return result
final = calculate(5, 3)
print(f"最终返回: {final}")
# 第1次执行
# 计算结果: 8
# 第2次执行
# 计算结果: 8
# 最终返回: 8
```

13 • 单项选择

关于 functools.wraps ，以下说法正确的是：

A. 用于创建装饰器

B. 用于保留原函数的元数据

C. 用于包装函数

D. 用于执行函数

答案 B. 用于保留原函数的元数据

解析 functools.wraps 用于保留原函数的元数据（如函数名、文档字符串等）。

示例：

```
import functools
# 不使用wraps
def decorator_without_wraps(func):
    def wrapper(*args, **kwargs):
        """wrapper的文档字符串"""
        print("装饰器执行")
        return func(*args, **kwargs)
    return wrapper
# 使用wraps
def decorator_with_wraps(func):
    @functools.wraps(func)
    # 保留原函数元数据
    def wrapper(*args, **kwargs):
        """wrapper的文档字符串"""
        print("装饰器执行")
        return func(*args, **kwargs)
    return wrapper
@decorator_without_wraps
def func1():
    """func1的文档字符串"""
    pass
@decorator_with_wraps
def func2():
    """func2的文档字符串"""
    pass
print(f"func1.__name__: {func1.__name__}")
# wrapper
print(f"func1.__doc__: {func1.__doc__}")
# wrapper的文档字符串
print(f"func2.__name__: {func2.__name__}")
# func2
print(f"func2.__doc__: {func2.__doc__}")
# func2的文档字符串
知识拓展：wraps保留的元数据
import functools
def my_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@my_decorator
def example_func(x, y):
    """这是一个示例函数"""
    return x + y
# wraps保留了这些属性：
print(f"__name__: {example_func.__name__}")
# example_func
```

```
print(f"__doc__: {example_func.__doc__}")
# 这是一个示例函数
print(f"__module__: {example_func.__module__}")
# __main__
print(f"__annotations__: {example_func.__annotations__}")
# {}
print(f"__qualname__: {example_func.__qualname__}")
# example_func
```

14 • 单项选择

以下代码的输出是什么？

```
def func(a, b=2, *args, **kwargs):
    return (a, b, args, kwargs)
result = func(1, 3, 4, 5, x=6, y=7)
print(len(result[2]))
```

A. 2

B. 3

C. 4

D. 5

答案 A. 2

解析 *args 接收位置参数 (4, 5) ，因此 args 是一个包含2个元素的元组，长度为2。

示例：

```
def func(a, b=2, *args, **kwargs):
    return (a, b, args, kwargs)
result = func(1, 3, 4, 5, x=6, y=7)
print(result)
# (1, 3, (4, 5), {'x': 6, 'y': 7})
print(f"a = {result[0]}")
# 1
print(f"b = {result[1]}")
# 3
print(f"args = {result[2]}")
# (4, 5)
print(f"kwargs = {result[3]}")
# {'x': 6, 'y': 7}
print(f"len(args) = {len(result[2])}")
# 2
```

知识拓展：参数解包详解

```
# 各种参数类型
def full_params(a, b=2, *args, c, d=4, **kwargs):
    print(f"必需参数 a: {a}")
    print(f"默认参数 b: {b}")
    print(f"可变位置参数 *args: {args}")
    print(f"keyword-only参数 c: {c}")
    print(f"keyword-only默认参数 d: {d}")
    print(f"可变关键字参数 **kwargs: {kwargs}")
# 调用示例
full_params(1, 2, 3, 4, c=5, d=6, x=7, y=8)
# 必需参数 a: 1
```

```
# 默认参数 b: 2
# 可变位置参数 *args: (3, 4)
# keyword-only参数 c: 5
# keyword-only默认参数 d: 6
# 可变关键字参数 **kwargs: {'x': 7, 'y': 8}
# 参数解包
args_tuple = (10, 20, 30)
kwargs_dict = {'c': 50, 'e': 60}
full_params(*args_tuple, **kwargs_dict)
# 必需参数 a: 10
# 默认参数 b: 20
# 可变位置参数 *args: (30,)
# keyword-only参数 c: 50
# keyword-only默认参数 d: 4
# 可变关键字参数 **kwargs: {'e': 60}
```

15 • 单项选择

关于关键字参数，以下说法错误的是：

A. 可以指定参数名传递

B. 必须在位置参数之后

C. 可以有默认值

D. 必须按顺序传递

答案 D. 必须按顺序传递

解析 关键字参数不必按顺序传递，可以按任意顺序指定参数名。

示例：

```
def greet(name, age, city):
    return f"{name}, {age}岁, 来自{city}"
# 按顺序传递
result1 = greet("Alice", 25, "北京")
print(result1)
# Alice, 25岁, 来自北京
# 使用关键字参数，不按顺序
result2 = greet(city="上海", name="Bob", age=30)
print(result2)
# Bob, 30岁, 来自上海
# 混合使用（位置参数必须在关键字参数前）
result3 = greet("Charlie", city="广州", age=28)
print(result3)
# Charlie, 28岁, 来自广州
```

知识拓展：仅限关键字参数

```
# 使用*强制后面的参数必须用关键字传递
def create_user(name, *, age, email, active=True):
    return {
        'name': name,
        'age': age,
        'email': email,
        'active': active
    }
# 正确调用
user1 = create_user("Alice", age=25, email="alice@example.com")
```

```
print(user1)
# 错误调用（会报错）
try:
    user2 = create_user("Bob", 30, "bob@example.com")
except TypeError as e:
    print(f"错误: {e}")
# 错误: create_user() takes 1 positional argument but 3 were given
# Python 3.8+ 仅限位置参数
def divide(a, b, /):
    """/ 前面的参数只能按位置传递"""
    return a / b
print(divide(10, 2))
# 5.0
# 错误调用
try:
    print(divide(a=10, b=2))
except TypeError as e:
    print(f"错误: {e}")
```

16 • 单项选择

Pandas中，以下哪个方法用于数据去重？

A. unique()

B. drop_duplicates()

C. remove_duplicates()

D. distinct()

答案 B. drop_duplicates()

解析 drop_duplicates() 方法用于删除DataFrame中的重复行。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 2, 3, 3, 3],
    'B': [10, 20, 20, 30, 30, 40]
})
print("原始数据：")
print(df)
# A B
# 0 1 10
# 1 2 20
# 2 2 20
# 3 3 30
# 4 3 30
# 5 3 40
# 删除完全重复的行
df_no_dup = df.drop_duplicates()
print("\n删除重复行后：")
print(df_no_dup)
# A B
# 0 1 10
# 1 2 20
# 3 3 30
```

```
# 5 3 40
# 基于特定列去重
df_no_dup_A = df.drop_duplicates(subset=['A'])
print("\n基于列A去重：")
print(df_no_dup_A)
# A B
# 0 1 10
# 1 2 20
# 3 3 30
# 保留最后一个重复项
df_keep_last = df.drop_duplicates(keep='last')
print("\n保留最后一个重复项：")
print(df_keep_last)
知识拓展：去重的高级用法
import pandas as pd
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Alice', 'Charlie'],
    'age': [25, 30, 25, 35],
    'score': [85, 90, 85, 88]
})
# keep参数：'first'（默认）、'last'、False
print(df.drop_duplicates(keep='first'))
# 保留第一个
print(df.drop_duplicates(keep='last'))
# 保留最后一个
print(df.drop_duplicates(keep=False))
# 删除所有重复
# inplace参数：直接修改原DataFrame
df.drop_duplicates(inplace=True)
# 查找重复行
duplicated = df.duplicated()
print(f"重复行：\n{duplicated}")
# 统计重复次数
from collections import Counter
dup_count = df['name'].value_counts()
print(f"\n各值出现次数：\n{dup_count}")
```

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 2, 3]})
print(len(df['A'].unique()))
```

A. 2

B. 3

C. 4

D. 1

答案 B. 3

解析 unique() 返回Series中的唯一值数组， [1, 2, 3] 的长度为3。

示例：

```
import pandas as pd
s = pd.Series([1, 2, 2, 3, 3, 3])
# 获取唯一值
unique_values = s.unique()
print(f"唯一值: {unique_values}")
# [1 2 3]
print(f"长度: {len(unique_values)}")
# 3
# nunique()直接返回唯一值的数量
print(f"唯一值数量: {s.nunique()}")
# 3
# value_counts()统计每个值的出现次数
print("\n值计数：")
print(s.value_counts())
# 3 3
# 2 2
# 1 1
知识拓展：unique vs nunique vs value_counts
import pandas as pd
import numpy as np
data = pd.Series([1, 2, 2, 3, 3, 3, np.nan, np.nan])
# unique(): 返回唯一值数组（包含NaN）
print(f"unique: {data.unique()}")
# [1.0 2.0 3.0 nan]
# nunique(): 返回唯一值数量（默认不计NaN）
print(f"nunique (不计NaN): {data.nunique()}")
# 3
print(f"nunique (计NaN): {data.nunique(dropna=False)}")
# 4
# value_counts(): 返回每个值的计数（默认不计NaN）
print("\nvalue_counts:")
print(data.value_counts())
# 3.0 3
# 2.0 2
# 1.0 1
print("\nvalue_counts (计NaN):")
print(data.value_counts(dropna=False))
# 3.0 3
# NaN 2
# 2.0 2
# 1.0 1
# DataFrame的unique操作
df = pd.DataFrame({
    'A': [1, 2, 2, 3],
    'B': [4, 5, 5, 6]
})
print("\nDataFrame唯一值：")
for col in df.columns:
    print(f"{col}: {df[col].unique()}")
```

18 • 单项选择

Pandas中， axis=0 和 axis=1 分别表示：

A. 行和列
B. 列和行
C. 横向和纵向
D. 上下和左右

答案 A. 行和列

解析 在Pandas中， axis=0 表示沿着行的方向（列操作）， axis=1 表示沿着列的方向（行操作）。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
print("原始DataFrame：")
print(df)
# A B C
# 0 1 4 7
# 1 2 5 8
# 2 3 6 9
# axis=0: 沿着行方向，对每列进行操作
print("\naxis=0 (对每列求和):")
print(df.sum(axis=0))
# A 6
# B 15
# C 24
# axis=1: 沿着列方向，对每行进行操作
print("\naxis=1 (对每行求和):")
print(df.sum(axis=1))
# 0 12
# 1 15
# 2 18
知识拓展：axis参数的记忆技巧和应用
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
# 记忆技巧：axis=0是"行索引"方向，操作结果是列
# axis=1是"列索引"方向，操作结果是行
# 各种操作的axis参数
print("mean (axis=0):", df.mean(axis=0))
# 每列的平均值
print("mean (axis=1):", df.mean(axis=1))
# 每行的平均值
print("\nmax (axis=0):", df.max(axis=0))
# 每列的最大值
print("max (axis=1):", df.max(axis=1))
# 每行的最大值
```

```
# 删除操作
df2 = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
# axis=0: 删除行
df_drop_row = df2.drop(0, axis=0)
print("\n删除第0行：")
print(df_drop_row)
# axis=1: 删除列
df_drop_col = df2.drop('B', axis=1)
print("\n删除B列：")
print(df_drop_col)
# 拼接操作
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
print("\naxis=0拼接（垂直）：")
print(pd.concat([df1, df2], axis=0))
print("\naxis=1拼接（水平）：")
print(pd.concat([df1, df2], axis=1))
```

19 • 单项选择

关于Pandas的 concat() 方法，以下说法正确的是：

A. 只能纵向拼接

B. 只能横向拼接

C. 可以指定拼接方向

D. 不能拼接多个DataFrame

答案 C. 可以指定拼接方向

解析 concat() 函数可以通过 axis 参数控制拼接方向： axis=0 垂直拼接， axis=1 水平拼接。

示例：

```
import pandas as pd
df1 = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
})
df2 = pd.DataFrame({
    'A': [5, 6],
    'B': [7, 8]
})
# axis=0: 垂直拼接（默认）
result_vertical = pd.concat([df1, df2], axis=0)
print("垂直拼接 (axis=0):")
print(result_vertical)
# A B
# 0 1 3
# 1 2 4
# 0 5 7
# 1 6 8
```

```
# axis=1: 水平拼接
result_horizontal = pd.concat([df1, df2], axis=1)
print("\n水平拼接 (axis=1):")
print(result_horizontal)
# A B A B
# 0 1 3 5 7
# 1 2 4 6 8
# 重置索引
result_reset = pd.concat([df1, df2], axis=0, ignore_index=True)
print("\n重置索引后:")
print(result_reset)
# A B
# 0 1 3
# 1 2 4
# 2 5 7
# 3 6 8
知识拓展：concat的高级用法
import pandas as pd
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'B': [5, 6], 'C': [7, 8]})
# join参数：inner（交集）或outer（并集，默认）
print("inner join:")
print(pd.concat([df1, df2], join='inner'))
# B
# 0 3
# 1 4
# 0 5
# 1 6
print("\nouter join (默认):")
print(pd.concat([df1, df2], join='outer'))
# A B C
# 0 1.0 3.0 NaN
# 1 2.0 4.0 NaN
# 0 NaN 5.0 7.0
# 1 NaN 6.0 8.0
# keys参数：添加层次化索引
result = pd.concat([df1, df2], keys=['df1', 'df2'])
print("\n带keys的拼接:")
print(result)
# Series的拼接
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
print("\nSeries拼接:")
print(pd.concat([s1, s2]))
# 0 1
# 1 2
# 2 3
# 0 4
# 1 5
# 2 6
```

20 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
result = pd.concat([s1, s2])
print(len(result))
```

A. 2

B. 3

C. 6

D. 9

答案 C. 6

解析 两个长度为3的Series拼接后，总长度为6。

示例：

```
import pandas as pd
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
# 拼接Series
result = pd.concat([s1, s2])
print(result)
# 0 1
# 1 2
# 2 3
# 0 4
# 1 5
# 2 6
print(f"\n拼接后长度: {len(result)}")
# 6
# 重置索引
result_reset = pd.concat([s1, s2], ignore_index=True)
print("\n重置索引后:")
print(result_reset)
# 0 1
# 1 2
# 2 3
# 3 4
# 4 5
# 5 6
```

知识拓展：Series拼接的多种方式

```
import pandas as pd
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['d', 'e', 'f'])
# 方法1: pd.concat()
result1 = pd.concat([s1, s2])
print("concat方式:")
print(result1)
# 方法2: append() (已弃用，但可能在旧代码中见到)
# result2 = s1.append(s2) # Deprecated
# 方法3: 使用list合并再创建
```

```
result3 = pd.Series(list(s1) + list(s2))
print("\nlist合并方式:")
print(result3)
# 多个Series拼接
s3 = pd.Series([7, 8, 9])
result_multi = pd.concat([s1, s2, s3], ignore_index=True)
print("\n多个Series拼接:")
print(result_multi)
# 0 1
# 1 2
# 2 3
# 3 4
# 4 5
# 5 6
# 6 7
# 7 8
# 8 9
```

21 • 单项选择

pathlib.Path 对象的 absolute() 和 resolve() 的区别是：

A. 完全相同
B. resolve() 会解析符号链接
C. absolute() 会解析符号链接
D. 两者都不解析符号链接

答案 B. resolve() 会解析符号链接

解析 resolve() 会解析符号链接并返回绝对路径，而 absolute() 只是返回绝对路径但不解析符号链接。

示例：

```
from pathlib import Path
# 相对路径
p = Path('docs/../config.ini')
print(f"原始路径: {p}")
print(f"absolute(): {p.absolute()}")
print(f"resolve(): {p.resolve()}")
```

知识拓展：路径解析的各种方法详见第1-3套相关题目。

22 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('.')
print(p.is_dir())
```

A. False
B. True
C. None
D. 报错

答案 B. True

解析 Path('.') 表示当前目录， is_dir() 返回True。

23 • 单项选择

Path.home() 方法返回的是：

A. 当前目录
B. 用户主目录
C. 根目录
D. 临时目录

答案 B. 用户主目录

解析 Path.home() 返回当前用户的主目录。

24 • 单项选择

关于 Path.match() 方法，以下说法正确的是：

A. 用于正则表达式匹配
B. 用于glob模式匹配
C. 用于精确匹配
D. 用于模糊匹配

答案 B. 用于glob模式匹配

解析 match() 方法用于glob模式匹配。

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('/a/b/c.txt')
print(len(p.parents))
```

A. 1
B. 2
C. 3
D. 4

答案 C. 3

解析 /a/b/c.txt 有3个父目录： /a/b 、 /a 和 / 。

26 • 单项选择

关于 sorted() 函数，以下说法正确的是：

A. 会修改原列表
B. 返回新列表
C. 不能指定排序键
D. 只能升序排序

答案 B. 返回新列表

解析 sorted() 函数返回一个新的排序列表，不修改原列表。

27 • 单项选择

以下代码的输出是什么？

```
data = [('a', 3), ('b', 1), ('c', 2)]
result = sorted(data, key=lambda x: x[1])
print(result[0])
```

A. ('a', 3)
B. ('b', 1)
C. ('c', 2)
D. 1

答案 B. ('b', 1)

解析 按照每个元素的第二个值排序，最小的是 ('b', 1) 。

28 • 单项选择

关于集合推导式，以下说法正确的是：

A. 使用 []
B. 使用 {}
C. 使用 ()
D. 使用 <>

答案 B. 使用 {}

解析 集合推导式使用大括号 {} 。

29 • 单项选择

以下代码的输出是什么？

```
result = {x % 3 for x in range(10)}
print(len(result))
```

A. 3
B. 10
C. 5
D. 7

答案 A. 3

解析 x % 3 对于0-9的结果只能是0、1、2，集合去重后有3个元素。

30 • 单项选择

关于 itertools 模块，以下说法正确的是：

A. 提供迭代器工具
B. 包含 chain() 、 cycle() 等函数
C. 用于高效循环
D. 以上都正确

答案 D. 以上都正确

解析 itertools 模块提供了各种迭代器工具，所有选项都正确。
（题21-30的详细示例代码和知识拓展可参考第1-3套相关题目的详细解析）
（这些题目的概念在第1-3套中已有详细解释，可参考相关套题的详细示例）

二、不定项选择题

31 • 不定项选择

关于Python的组合和继承，以下说法正确的有：

A. 组合是"has-a"关系

B. 继承是"is-a"关系
C. 组合比继承更灵活
D. 应该优先使用继承

答案 A. 组合是"has-a"关系；B. 继承是"is-a"关系；C. 组合比继承更灵活

解析 应该优先使用组合而非继承（Composition over Inheritance）是面向对象设计的重要原则。

示例：

```
# 继承方式（不推荐）
class Vehicle:
    def move(self):
        pass
class FlyingVehicle(Vehicle):
    def fly(self):
        return "飞行"
class SwimmingVehicle(Vehicle):
    def swim(self):
        return "游泳"
class FlyingSwimmingVehicle(FlyingVehicle, SwimmingVehicle):
    # 多重继承复杂，难以维护
    pass
# 组合方式（推荐）
class FlyingAbility:
    def fly(self):
        return "飞行"
class SwimmingAbility:
    def swim(self):
        return "游泳"
class Vehicle:
    def __init__(self):
        self.abilities = []
    def add_ability(self, ability):
        self.abilities.append(ability)
    def perform_abilities(self):
        for ability in self.abilities:
            if hasattr(ability, 'fly'):
                print(ability.fly())
            if hasattr(ability, 'swim'):
                print(ability.swim())
# 灵活组合
vehicle = Vehicle()
vehicle.add_ability(FlyingAbility())
vehicle.add_ability(SwimmingAbility())
vehicle.perform_abilities()
```

知识拓展：组合vs继承的优缺点：

- 组合优点：更灵活、更好的封装、避免类层次爆炸
- 继承优点：代码复用、is-a关系明确

2-5. 答案：

答案：ABCD ABCD ABCD ABCD

这些题目涉及魔法方法、生成器特性、迭代器创建方式等，在第1-3套中已有详细解释，可参考相关套题。

核心要点：
- 魔法方法：__str__ 、 __repr__ 、 __len__ 、 __getitem__ 等
- 生成器：惰性求值、节省内存、只能遍历一次
- 迭代器：实现 __iter__ 和 __next__ 、生成器表达式、iter()函数

32 • 不定项选择

以下哪些是Python的数据模型方法（魔法方法）：

A. __add__()
B. __mul__()
C. __contains__()
D. __missing__()

答案 A. __add__()；B. __mul__()；C. __contains__()；D. __missing__()

解析 （原卷合并作答）

33 • 不定项选择

关于Python的属性访问，以下说法正确的有：

A. __getattr__() 在属性不存在时调用
B. __setattr__() 在设置属性时调用
C. __delattr__() 在删除属性时调用
D. __getattribute__() 在访问任何属性时调用

答案 A. __getattr__() 在属性不存在时调用；B. __setattr__() 在设置属性时调用；C. __delattr__() 在删除属性时调用；D. __getattribute__() 在访问任何属性时调用

解析 （原卷合并作答）

34 • 不定项选择

关于生成器的特性，以下说法正确的有：

A. 惰性计算
B. 只能迭代一次
C. 节省内存
D. 可以表示无限序列

答案 A. 惰性计算；B. 只能迭代一次；C. 节省内存；D. 可以表示无限序列

解析 （原卷合并作答）

35 • 不定项选择

以下哪些是创建迭代器的方法：

A. 实现 __iter__() 和 __next__()
B. 使用生成器函数
C. 使用生成器表达式
D. 使用 iter() 函数

答案 A. 实现 __iter__() 和 __next__()；B. 使用生成器函数；C. 使用生成器表达式；D. 使用 iter() 函数

解析 （原卷合并作答）

36 • 不定项选择

关于装饰器的高级用法，以下说法正确的有：

A. 可以装饰类
B. 可以叠加使用
C. 可以带参数
D. 可以访问被装饰函数的属性

答案 A. 可以装饰类；B. 可以叠加使用；C. 可以带参数；D. 可以访问被装饰函数的属性

解析 装饰器的所有选项都是正确的应用场景。
（详细示例可参考第1-3套装饰器相关题目）

37 • 不定项选择

关于闭包的应用场景，以下说法正确的有：

A. 实现装饰器
B. 实现回调函数
C. 保存状态
D. 实现工厂函数

答案 A. 实现装饰器；B. 实现回调函数；C. 保存状态；D. 实现工厂函数

解析 所有选项都是闭包的常见应用场景。
（详细示例可参考第1-3套闭包相关题目）

38 • 不定项选择

以下哪些是Pandas的数据类型：

A. int64
B. float64
C. object
D. datetime64

答案 A. int64；B. float64；C. object；D. datetime64

解析 所有选项都是Pandas中的数据类型。
（详细示例可参考第1-3套Pandas相关题目）

39 • 不定项选择

关于Pandas的数据选择，以下说法正确的有：

A. df['col'] 选择列
B. df[0] 选择第一行
C. df[0:3] 选择前三行
D. df.col 选择列

答案 A. df['col'] 选择列；C. df[0:3] 选择前三行；D. df.col 选择列

解析 df[0] 会报错，应该使用 df.iloc[0] 或 df.loc[0] 。
（详细示例可参考第1-3套Pandas索引相关题目）

40 • 不定项选择

Pandas中，以下哪些方法用于数据转换：

A. astype()
B. apply()
C. map()
D. transform()

答案 A. astype()；B. apply()；C. map()；D. transform()

解析 所有选项都是数据转换的常用方法。

（详细示例可参考第1-3套Pandas数据转换相关题目）

41 • 不定项选择

关于 pathlib 的文件操作，以下说法正确的有：

A. read_text() 读取文本文件

B. write_text() 写入文本文件

C. read_bytes() 读取二进制文件

D. write_bytes() 写入二进制文件

答案 A. read_text() 读取文本文件；B. write_text() 写入文本文件；C. read_bytes() 读取二进制文件；D. write_bytes() 写入二进制文件

解析 pathlib 的所有选项都是正确的优势。

（详细示例可参考第1-3套pathlib相关题目）

42 • 不定项选择

Path 对象的以下哪些方法会创建文件或目录：

A. mkdir()

B. touch()

C. exists()

D. rename()

答案 A. mkdir()；B. touch()

解析 exists() 用于检查路径是否存在， mkdir() 用于创建目录； rename() 是重命名而非移动文件。

（详细示例可参考第1-3套pathlib相关题目）

43 • 不定项选择

关于推导式的性能，以下说法正确的有：

A. 通常比等价的for循环快

B. 代码更简洁

C. 可读性更好

D. 适合简单的转换操作

答案 A. 通常比等价的for循环快；B. 代码更简洁；D. 适合简单的转换操作

解析 推导式的可读性不一定更好，取决于复杂度。

（详细示例可参考第1-3套推导式相关题目）

44 • 不定项选择

以下哪些是Python的内置迭代工具：

A. enumerate()

B. zip()

C. reversed()

D. range()

答案 A. enumerate()；B. zip()；C. reversed()；D. range()

解析 所有选项都是Python内置的迭代工具。
（详细示例可参考第1-3套函数式编程相关题目）

45 • 不定项选择

关于函数式编程，以下说法正确的有：

A. 避免副作用
B. 使用纯函数
C. 数据不可变
D. Python完全支持函数式编程

答案 A. 避免副作用；B. 使用纯函数；C. 数据不可变

解析 Python支持函数式编程特性，但不是纯函数式语言。
（详细示例可参考第1-3套函数式编程相关题目）

三、填空题

46 • 填空 / 简答

Python中，使用 _____ 可以创建抽象基类，使用 _____ 装饰器标记抽象方法。

答案 abc.ABC （或

解析 （或 ABC ），
），@abstractmethod
抽象基类用于定义接口。
示例：

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
    @abstractmethod
    def move(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "汪汪"
    def move(self):
        return "跑"
# 无法实例化抽象类
# animal = Animal() # TypeError
dog = Dog()
print(dog.make_sound())
# 汪汪
```

知识拓展：抽象方法强制子类必须实现，否则无法实例化。

47 • 填空 / 简答

生成器的三个主要方法是、和 _____。

答案 __next__() ， send() ， close()

解析 生成器的三个核心方法。

示例：

```
def counter():
    total = 0
    while True:
        increment = yield total
        if increment:
            total += increment
gen = counter()
print(next(gen))
# 0 - __next__()
print(gen.send(5))
# 5 - send()
print(gen.send(10))
# 15
gen.close()
# close()
# print(next(gen)) # StopIteration
```

知识拓展： send() 用于协程， close() 提前终止生成器。

48 • 填空 / 简答

Pandas中，_____ 方法用于数据透视表，_____ 方法用于交叉表。

答案 pivot_table() ， crosstab()

解析 Pandas数据透视表方法。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'date': ['2023-01', '2023-01', '2023-02', '2023-02'],
    'product': ['A', 'B', 'A', 'B'],
    'sales': [100, 200, 150, 250]
})
# pivot_table
pt = df.pivot_table(values='sales',
                    index='date',
                    columns='product')
print(pt)
# crosstab
ct = pd.crosstab(df['date'], df['product'],
                 values=df['sales'], aggfunc='sum')
print(ct)
```

知识拓展： pivot_table() 功能更强， crosstab() 专用于频数统计。

49 • 填空 / 简答

使用 _____ 函数可以对可迭代对象进行排序，使用 _____ 参数可以指定排序键。

答案 sorted() ， key

解析 排序函数和关键字参数。

示例：

```
# 基本排序
```

```
data = [3, 1, 4, 1, 5, 9, 2, 6]
print(sorted(data))
# [1, 1, 2, 3, 4, 5, 6, 9]
# 使用key指定排序规则
words = ['apple', 'pie', 'zoo', 'a']
print(sorted(words, key=len))
# ['a', 'pie', 'zoo', 'apple']
# 复杂排序
students = [
    {'name': 'Alice', 'age': 25, 'score': 85},
    {'name': 'Bob', 'age': 23, 'score': 90},
    {'name': 'Charlie', 'age': 25, 'score': 80}
]
# 按分数排序
print(sorted(students, key=lambda x: x['score']))
# 按多个条件排序
print(sorted(students, key=lambda x: (x['age'], -x['score'])))
```

知识拓展： key 参数接受一个函数，用于提取比较键。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法返回所有匹配的文件，_____ 方法递归搜索。

答案 glob() ， rglob()

解析 Path对象的文件搜索方法。

示例：

```
from pathlib import Path
project = Path('/home/user/project')
# glob - 非递归搜索
for file in project.glob('*.py'):
    print(file)
# 只搜索当前目录
# rglob - 递归搜索
for file in project.rglob('*.py'):
    print(file)
# 搜索所有子目录
# 复杂模式
for file in project.rglob('**/test_*.py'):
    print(file)
# 找到所有test_开头的py文件
```

知识拓展： rglob('pattern') 等价于 glob('**/pattern') 。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Descriptor:
    def __init__(self, name):
        self.name = name
```

```
    def __get__(self, instance, owner):
        if instance is None:
            return self
        return instance.__dict__.get(self.name, None)
    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("值不能为负")
        instance.__dict__[self.name] = value
class Product:
    price = Descriptor('price')
    def __init__(self, price):
        self.price = price
p = Product(100)
print(p.price)
p.price = 200
print(p.price)
```

问题：

- (1) 两个 print 语句的输出分别是什么？
- (2) 这段代码实现了什么功能？
- (3) 如果执行 p.price = -10 会发生什么？

答案 描述符

- (1) 输出：100, 200
- (2) 实现了描述符协议，用于属性访问控制和验证
- (3) 会抛出 ValueError 异常，提示"值不能为负"

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
from itertools import islice
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
fib = fibonacci()
result = list(islice(fib, 10))
print(result)
print(next(fib))
```

问题：

- (1) result 的内容是什么？
- (2) 最后一个 print 输出什么？
- (3) 为什么这个生成器可以表示无限序列？

答案 • (1) [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

- (2) 输出：55（第11个斐波那契数）
- (3) 使用 while True 无限循环，通过 yield 按需生成值

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=5),
    'value': [10, 20, 30, 40, 50]
})
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
result = df.resample('2D').sum()
print(result)
```

问题：

- (1) resample('2D') 的作用是什么？
- (2) result 会有几行数据？
- (3) 为什么需要先设置索引为日期？

答案 • (1) resample('2D') 按2天为间隔重新采样数据
- (2) 3行（5天按2天分组：第1-2天、第3-4天、第5天）
- (3) 时间序列操作需要DatetimeIndex

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
def backup_files(source, backup_dir, extensions):
    source_path = Path(source)
    backup_path = Path(backup_dir)
    backup_path.mkdir(parents=True, exist_ok=True)
    for ext in extensions:
        for file in source_path.glob(f'*.{ext}'):
            dest = backup_path / file.name
            dest.write_bytes(file.read_bytes())
            yield file.name
files = list(backup_files('.', 'backup', ['py', 'txt']))
print(f"备份了{len(files)} 个文件")
```

问题：

- (1) 这个函数备份了哪些文件？
- (2) 为什么使用 read_bytes() 和 write_bytes() ？
- (3) 使用生成器的好处是什么？

答案 • (1) 备份当前目录下所有.py和.txt文件
- (2) 使用字节模式可以处理任何文件类型，包括二进制文件
- (3) 生成器可以逐个处理文件，节省内存，并提供进度反馈

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
from functools import lru_cache
@lru_cache(maxsize=None)
def expensive_function(n):
    print(f"计算{n}")
    return n ** 2
print(expensive_function(5))
print(expensive_function(10))
print(expensive_function.cache_info())
```

问题：

- (1) "计算 5"会打印几次？
- (2) lru_cache 的作用是什么？
- (3) maxsize=None 表示什么？

答案 缓存

- (1) 只打印1次（第二次从缓存获取）
- (2) lru_cache 实现最近最少使用缓存，避免重复计算
- (3) maxsize=None 表示缓存大小无限制

解析

五、程序设计题

56 • 填空 / 简答

设计一个购物车系统，包含以下类：

Product类：

类：

- 属性：名称、价格、库存
- 方法： __str__() 、 __eq__() （按名称比较）

ShoppingCart类：

类：

- 属性：商品字典（商品：数量）
- 方法：

add_item(product, quantity) ：添加商品

remove_item(product) ：移除商品

update_quantity(product, quantity) ：更新数量

total_price ：使用 @property 计算总价

__len__() ：返回商品种类数

__iter__() ：使购物车可迭代

要求：

- 检查库存是否足够
- 使用列表推导式筛选价格大于某值的商品
- 实现上下文管理器（ __enter__ 和 __exit__ ）

答案

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
```

```
        self.stock = stock
    def __str__(self):
        return f"{self.name} - ¥{self.price} (库存: {self.stock})"
    def __eq__(self, other):
        if isinstance(other, Product):
            return self.name == other.name
        return False
    def __hash__(self):
        return hash(self.name)
class ShoppingCart:
    def __init__(self):
        self.items = {}
    def __enter__(self):
        print("开始购物")
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("结束购物")
        return False
    def add_item(self, product, quantity):
        if quantity > product.stock:
            raise ValueError(f"{product.name} 库存不足")
        if product in self.items:
            self.items[product] += quantity
        else:
            self.items[product] = quantity
        product.stock -= quantity
    print(f"已添加{quantity} 个{product.name}")
def remove_item(self, product):
    if product in self.items:
        quantity = self.items[product]
        product.stock += quantity
        del self.items[product]
        print(f"已移除{product.name}")
def update_quantity(self, product, quantity):
    if product in self.items:
        old_quantity = self.items[product]
        diff = quantity - old_quantity
        if diff > product.stock:
            raise ValueError(f"{product.name} 库存不足")
        self.items[product] = quantity
        product.stock -= diff
@property
def total_price(self):
    return sum(product.price * quantity
               for product, quantity in self.items.items())
def __len__(self):
    return len(self.items)
def __iter__(self):
```

```
    return iter(self.items.items())
def filter_by_price(self, min_price):
    return [product for product, qty in self.items.items()
            if product.price > min_price]
# 使用示例
p1 = Product("Python书籍", 89, 10)
p2 = Product("数据分析", 79, 5)
p3 = Product("机器学习", 99, 8)
with ShoppingCart() as cart:
    cart.add_item(p1, 2)
    cart.add_item(p2, 1)
    cart.add_item(p3, 1)
    print(f"\n购物车商品数: {len(cart)}")
    print(f"总价: ¥{cart.total_price}")
    print("\n购物车内容:")
    for product, quantity in cart:
        print(f"{product.name} x {quantity}")
    expensive = cart.filter_by_price(80)
    print(f"\n价格大于80的商品: {[p.name for p in expensive]}")
```

解析

57 • 填空 / 简答

编写一个配置文件管理器，要求：

1. ConfigManager 类：

- 使用单例模式
- 从JSON/YAML文件加载配置
- 支持嵌套配置访问（如 config.database.host ）
- 使用 @property 实现配置项访问

2. 装饰器 @require_config(key) ：

- 检查配置项是否存在
- 不存在则抛出异常

3. 使用 pathlib 处理配置文件路径
4. 实现配置热加载（文件修改时自动重新加载）

答案

```
import json
from pathlib import Path
from functools import wraps
import time
class ConfigManager:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
    def __init__(self, config_file=None):
        if not hasattr(self, '_initialized'):
            self._config = {}
            self._config_file = None
```

```
            self._last_modified = None
            if config_file:
                self.load_config(config_file)
            self._initialized = True
    def load_config(self, config_file):
        path = Path(config_file)
        if not path.exists():
            raise FileNotFoundError(f"配置文件{config_file} 不存在")
        with path.open('r', encoding='utf-8') as f:
            self._config = json.load(f)
        self._config_file = path
        self._last_modified = path.stat().st_mtime
        print(f"已加载配置: {config_file}")
    def reload_if_modified(self):
        if self._config_file:
            current_mtime = self._config_file.stat().st_mtime
            if current_mtime > self._last_modified:
                print("检测到配置文件变化，重新加载...")
                self.load_config(self._config_file)
    def get(self, key, default=None):
        self.reload_if_modified()
        keys = key.split('.')
        value = self._config
        for k in keys:
            if isinstance(value, dict) and k in value:
                value = value[k]
            else:
                return default
        return value
    @property
    def config(self):
        self.reload_if_modified()
        return self._config
def require_config(key):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            config = ConfigManager()
            if config.get(key) is None:
                raise ValueError(f"缺少配置项: {key}")
            return func(*args, **kwargs)
        return wrapper
    return decorator
# 使用示例
config_data = {
    "database": {
        "host": "localhost",
        "port": 5432,
```

```
        "name": "mydb"
    },
    "app": {
        "debug": True,
        "version": "1.0"
    }
# 创建配置文件
config_file = Path('config.json')
config_file.write_text(json.dumps(config_data, indent=2))
# 使用配置管理器
config = ConfigManager('config.json')
print(f"数据库主机: {config.get('database.host')}")
print(f"应用版本: {config.get('app.version')}")
@require_config('database.host')
def connect_database():
    config = ConfigManager()
    host = config.get('database.host')
    print(f"连接到数据库: {host}")
connect_database()
```

解析

58 • 填空 / 简答

使用Pandas分析时间序列数据：

创建一个月的销售数据（日期、产品、销量、金额）：

- 按周统计销售额
- 计算移动平均（7天窗口）
- 找出销售高峰日
- 按产品分组计算增长率
- 使用 pivot_table() 创建产品-日期透视表
- 处理缺失日期（填充为0）
- 绘制销售趋势图（使用matplotlib）
- 导出分析报告

答案

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
# 创建一个月的销售数据
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=30, freq='D')
products = ['产品A', '产品B', '产品C']
data = []
for date in dates:
    for product in products:
        sales = np.random.randint(50, 200)
        amount = sales * np.random.uniform(10, 50)
        data.append({
            '日期': date,
```

```
            '产品': product,
            '销量': sales,
            '金额': amount
        })
df = pd.DataFrame(data)
df.set_index('日期', inplace=True)
print("原始数据预览:")
print(df.head(10))
# 按周统计销售额
weekly_sales = df.groupby([pd.Grouper(freq='W'), '产品'])['金额'].sum()
print("\n按周统计销售额:")
print(weekly_sales)
# 计算移动平均（7天窗口）
df_sorted = df.sort_index()
df_sorted['移动平均'] = df_sorted.groupby('产品')['金额'].transform(
    lambda x: x.rolling(window=7, min_periods=1).mean()
)
print("\n移动平均数据:")
print(df_sorted.head(10))
# 找出销售高峰日
daily_total = df.groupby(df.index)['金额'].sum()
peak_day = daily_total.idxmax()
peak_amount = daily_total.max()
print(f"\n销售高峰日: {peak_day.date()}, 金额: {peak_amount:.2f}")
# 按产品计算增长率
product_growth = df.groupby('产品')['金额'].apply(
    lambda x: (x.iloc[-7:].mean() - x.iloc[:7].mean()) / x.iloc[:7].mean() * 100
)
print("\n产品增长率（最后7天 vs 前7天）:")
print(product_growth)
# 创建数据透视表
pivot = df.pivot_table(
    values='金额',
    index=df.index.date,
    columns='产品',
    aggfunc='sum',
    fill_value=0
)
print("\n产品-日期透视表:")
print(pivot.head())
# 绘制销售趋势图
plt.figure(figsize=(12, 6))
for product in products:
    product_data = df[df['产品'] == product].groupby(df.index)['金额'].sum()
    plt.plot(product_data.index, product_data.values, label=product, marker='o')
plt.title('销售趋势图')
plt.xlabel('日期')
plt.ylabel('销售金额')
```

```
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('sales_trend.png')
print("\n趋势图已保存为 sales_trend.png")
# 导出分析报告
with pd.ExcelWriter('sales_report.xlsx') as writer:
    df.to_excel(writer, sheet_name='原始数据')
    weekly_sales.to_excel(writer, sheet_name='周度统计')
    pivot.to_excel(writer, sheet_name='透视表')
print("分析报告已导出为 sales_report.xlsx")
```

解析

59 • 填空 / 简答

编写一个Python代码质量检查工具：

1. analyze_file(filepath) ：
- 统计代码行数、注释行数、空行数
- 检测函数和类的数量
- 计算代码复杂度（简单统计if/for/while数量）
2. find_long_functions(directory, max_lines) ：
- 使用生成器查找超过指定行数的函数
- 返回函数名和行数
3. check_naming_convention(filepath) ：
- 检查变量和函数命名是否符合PEP8

要求：

- 使用 pathlib 遍历目录
- 使用正则表达式解析代码
- 使用装饰器记录分析时间
- 生成HTML格式的报告

答案

```
from pathlib import Path
import re
from functools import wraps
import time
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} 执行时间: {end-start:.4f}秒")
        return result
    return wrapper
@timer
def analyze_file(filepath):
    path = Path(filepath)
    if not path.exists():
```

```
        return None
    with path.open('r', encoding='utf-8') as f:
        lines = f.readlines()
    total_lines = len(lines)
    code_lines = 0
    comment_lines = 0
    blank_lines = 0
    for line in lines:
        stripped = line.strip()
        if not stripped:
            blank_lines += 1
        elif stripped.startswith('#'):
            comment_lines += 1
        else:
            code_lines += 1
    # 统计函数和类
    content = ''.join(lines)
    functions = len(re.findall(r'^\s*def\s+\w+', content, re.MULTILINE))
    classes = len(re.findall(r'^\s*class\s+\w+', content, re.MULTILINE))
    # 简单复杂度（if/for/while数量）
    complexity = (
        len(re.findall(r'\bif\b', content)) +
        len(re.findall(r'\bfor\b', content)) +
        len(re.findall(r'\bwhile\b', content))
    )
    return {
        'file': filepath,
        'total_lines': total_lines,
        'code_lines': code_lines,
        'comment_lines': comment_lines,
        'blank_lines': blank_lines,
        'functions': functions,
        'classes': classes,
        'complexity': complexity
    }
def find_long_functions(directory, max_lines=50):
    path = Path(directory)
    for py_file in path.rglob('*.py'):
        with py_file.open('r', encoding='utf-8') as f:
            content = f.read()
        # 简单查找函数定义
        pattern = r'def\s+(\w+)\s*\([^)]*\):'
        matches = re.finditer(pattern, content)
        for match in matches:
            func_name = match.group(1)
            start_pos = match.start()
            # 估算函数行数（简化版）
            func_content = content[start_pos:]
```

```
            lines = func_content.split('\n')
            func_lines = 0
            indent_level = None
            for line in lines[1:]:
                if line.strip():
                current_indent = len(line) - len(line.lstrip())
                if indent_level is None:
                indent_level = current_indent
                elif current_indent <= indent_level and line.strip():
                break
                func_lines += 1
            if func_lines > max_lines:
                yield {
                'file': str(py_file),
                'function': func_name,
                'lines': func_lines
                }
def check_naming_convention(filepath):
    path = Path(filepath)
    with path.open('r', encoding='utf-8') as f:
        content = f.read()
    issues = []
    # 检查函数名（应该是snake_case）
    func_pattern = r'def\s+([A-Z]\w+)\s*\('
    for match in re.finditer(func_pattern, content):
        issues.append(f"函数名{match.group(1)} 应使用snake_case")
    # 检查类名（应该是PascalCase）
    class_pattern = r'class\s+([a-z]\w+)\s*[:(]'
    for match in re.finditer(class_pattern, content):
        issues.append(f"类名{match.group(1)} 应使用PascalCase")
    return issues
def generate_html_report(analysis_results, output_file='report.html'):
html = """
<html>
<head>
    <title>代码质量报告</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
        th { background-color: #4CAF50; color: white; }
        tr:nth-child(even) { background-color: #f2f2f2; }
    </style>
</head>
<body>
    <h1>代码质量分析报告</h1>
    <table>
        <tr>
```

```
            <th>文件</th>
            <th>总行数</th>
            <th>代码行</th>
            <th>注释行</th>
            <th>空行</th>
            <th>函数数</th>
            <th>类数</th>
            <th>复杂度</th>
        </tr>
"""
for result in analysis_results:
    if result:
        html += f"""
        <tr>
            <td>{result['file']}</td>
            <td>{result['total_lines']}</td>
            <td>{result['code_lines']}</td>
            <td>{result['comment_lines']}</td>
            <td>{result['blank_lines']}</td>
            <td>{result['functions']}</td>
            <td>{result['classes']}</td>
            <td>{result['complexity']}</td>
        </tr>
        """
html += """
    </table>
</body>
</html>
"""
Path(output_file).write_text(html, encoding='utf-8')
print(f"HTML报告已生成: {output_file}")
# 使用示例
# results = []
# for py_file in Path('.').rglob('*.py'):
# result = analyze_file(py_file)
# if result:
# results.append(result)
# print(f"\n分析 {py_file}:")
# print(f" 代码行: {result['code_lines']}")
# print(f" 函数数: {result['functions']}")
# print(f" 复杂度: {result['complexity']}")
# generate_html_report(results)
# print("\n查找长函数:")
# for func_info in find_long_functions('.', max_lines=30):
# print(f" {func_info['function']} in {func_info['file']}: {func_info['lines']} 行")
```

解析

60 • 填空 / 简答

实现一个简单的数据处理管道框架：

1. Pipeline 类：

答案

```
from functools import wraps
import time
def pipeline_step(func):
    @wraps(func)
    def wrapper(data):
        start = time.time()
        input_size = len(list(data)) if hasattr(data, '__iter__') else 1
        result = func(data)
        end = time.time()
        output_size = len(list(result)) if hasattr(result, '__iter__') else 1
        print(f"步骤{func.__name__}:")
        print(f" 输入: {input_size} 项")
        print(f" 输出: {output_size} 项")
        print(f" 耗时: {end-start:.4f}秒")
        return result
    return wrapper
class Pipeline:
    def __init__(self):
        self.steps = []
    def add_step(self, step_func):
        self.steps.append(step_func)
        return self
    def execute(self, data):
        result = data
        for step in self.steps:
            result = step(result)
        return result
def filter_data(condition):
    @pipeline_step
    def filter_step(data):
        for item in data:
            if condition(item):
                yield item
    return filter_step
def transform_data(func):
    @pipeline_step
    def transform_step(data):
        for item in data:
            yield func(item)
    return transform_step
def aggregate_data(func):
    @pipeline_step
    def aggregate_step(data):
        return func(list(data))
    return aggregate_step
```

```
# 使用示例
print("=== 数据管道示例 ===\n")
# 原始数据
data = range(-10, 11)
print(f"原始数据: {list(data)}\n")
# 创建管道
pipeline = Pipeline()
result = (pipeline
    .add_step(filter_data(lambda x: x > 0))
    .add_step(transform_data(lambda x: x * 2))
    .add_step(transform_data(lambda x: x ** 2))
    .execute(data))
print(f"\n最终结果: {result}")
# 另一个示例：数据聚合
print("\n=== 聚合示例 ===\n")
data2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
pipeline2 = Pipeline()
total = (pipeline2
    .add_step(filter_data(lambda x: x % 2 == 0))
    .add_step(transform_data(lambda x: x * 3))
    .add_step(aggregate_data(sum))
    .execute(data2))
print(f"\n总和: {total}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：
  总分：100分

解析